# Architecture-Aware Evaluation of Range Query Processing: Mo's Algorithm vs GPU Parallelization

*A Project Component for*

**Parallel and Distributed Computing (UCS645)**

*By*


| Sr | Name | Roll No |
|---|---|---|
| 1 | Suryansh Sharma | 102316055 |
| 2 | Shorya Gupta | 102316062 |

*Under the guidance of*

**Dr. Saif Nalband**

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

PATIALA - 147004


**APRIL, 2026**

# Table of Contents


**1 Introduction** **1**

1.1 Background . . . . . 1

1.2 Introduction to Problem Statement . . . . . 1

**2 Literature Review** **3**

**3 Research Gaps** **4**

**4 Problem Formulation** **5**

**5 Objectives** **6**

**6 Methodology** **7**

6.1 Core Equation and `prev[]` Transformation . . . . . 7

6.2 Engine 1: Mo's Algorithm with Hilbert Ordering . . . . . 7

6.3 Engine 2: OpenMP + SIMD . . . . . 8

6.4 Engine 3: CUDA Warp-Level Parallelism . . . . . 8

6.5 Benchmarking Protocol . . . . . 8

**7 Datasets** **9**

**8 Results and Discussion** **10**

8.1 Execution Time . . . . . 10

8.2 Throughput and Speedup . . . . . 10

8.3 Range Sensitivity . . . . . 10

8.4 Thread Efficiency (OpenMP) . . . . . 10

8.5 GPU Transfer/Kernel Breakdown . . . . . 11

**References** **14**

# List of Figures


1 Speedup comparison at $Q = 100{,}000$ (medium ranges). . . . . 2

2 Speedup comparison at $Q = 100{,}000$ (medium ranges). . . . . 11

3 Execution time vs. number of queries (log scale). . . . . 12

4 Throughput comparison across methods (log scale). . . . . 12

5 Query size vs. performance across range classes. . . . . 13

6 OpenMP efficiency vs. thread count. . . . . 13

# List of Tables


1 Summary of Literature Review . . . . . 3

2 Dataset Generation Parameters . . . . . 9

3 Datasets . . . . . 9

4 Summary at $Q = 100{,}000$ (medium ranges, max_len=2048) . . . . . 10

# 1 Introduction

## 1.1 Background

Distinct elements in range queries are a fundamental primitive in analytics, indexing, and log mining. At large scales, naive per-query scans are slow and memory-inefficient, so the algorithm must be redesigned with hardware realities in mind.

This project implements three architecture-aware engines: a cache-friendly single-core Mo's algorithm, a multicore SIMD OpenMP engine based on a prev-array transformation, and a CUDA kernel that assigns one warp per query.

## 1.2 Introduction to Problem Statement

Given an immutable array $A[0..N-1]$ and a set of offline range queries $[L, R]$, compute:

$$\text{distinct}(L, R) = |\{A_i \mid L \leq i \leq R\}|$$

The challenge is to answer up to $Q = 100{,}000$ queries on arrays as large as $N = 1{,}000{,}000$ while respecting cache locality, memory bandwidth ceilings, SIMD lane utilization, and GPU warp behavior.

Figure 2 summarizes the relative speedups at the largest workload.

Key contributions of this work are:

- Hilbert-ordered Mo's algorithm to reduce cache churn for locality-heavy query sets.
- A *prev-array* formulation that transforms distinct counting into a sequential, SIMD-friendly scan.
- A CUDA kernel that assigns one warp per query with warp-level reduction to minimize synchronization overhead.

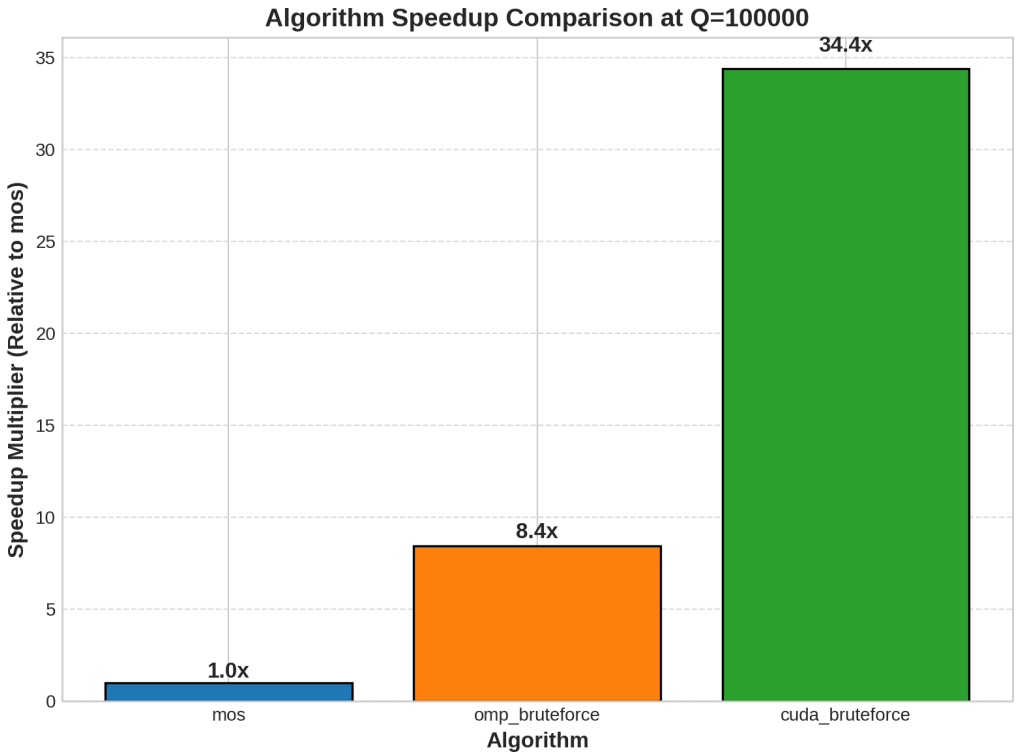


Figure 1: Speedup comparison at $Q = 100{,}000$ (medium ranges).

# 2 Literature Review

Table 1: Summary of Literature Review

| **References** | **Concepts Highlighted** | **Technique(s) used** | **Significance/ Proposal** | **Limitations** |
|---|---|---|---|---|
| Mo's algorithm (offline range queries) | Query reordering for locality | Hilbert curve ordering | Reduces window movement and cache churn for range queries | Single-threaded baseline; limited throughput at high $Q$ |
| SIMD + OpenMP parallel scans | Memory bandwidth and vectorization | Prev-array transformation; SIMD reduction | Converts irregular distinct counting into sequential, prefetch-friendly scans | Scaling limited by memory bandwidth beyond moderate threads |
| GPU warp-level reduction | Coalesced memory access and warp execution | One warp per query; shuffle reduction | High throughput for large query batches with favorable range geometry | Transfer overhead and warp occupancy constraints on small batches |

# 3 Research Gaps

Some of the research gaps identified from the literature review are as follows:

- ***Hardware-aware formulations***: many distinct-range solutions ignore memory hierarchy and SIMD/GPU execution models.
- ***Cross-architecture benchmarking***: few works present consistent, end-to-end comparisons across CPU single-core, multicore SIMD, and GPU warp-level designs.
- ***Range-geometry sensitivity***: limited analysis exists on how query length distributions affect throughput across architectures.

# 4 Problem Formulation

We are given an immutable integer array $A$ of size $N$ and a set of $Q$ offline range queries. For each query $[L, R]$, we must compute the number of distinct values within the subarray. The objective is to deliver high throughput and low latency under production-scale settings ($N = 1{,}000{,}000$, $Q = 100{,}000$), where repeated scanning of ranges becomes prohibitively expensive.

The problem is hardware-sensitive: naive methods perform irregular memory accesses and redundant checks, which is unfavorable for caches, SIMD vector lanes, and GPU warps. We therefore seek algorithmic reformulations that turn the work into sequential scans and coalesced access patterns. Performance is evaluated by query time, throughput (queries per second), speedup relative to Mo's baseline, and scaling efficiency on multicore CPUs.

# 5 Objectives

- Design and implement three optimized engines (Mo's, OpenMP SIMD, CUDA) for distinct-range queries.
- Quantify throughput and speedup across query volumes and range sizes.
- Analyze multicore scaling behavior and GPU transfer/kernel breakdowns.

# 6 Methodology

The methodology follows a reproducible pipeline: generate datasets, preprocess the array, execute three architecture-specific engines, and benchmark under identical inputs.

## 6.1 Core Equation and `prev[]` Transformation

We precompute a previous-occurrence array:

$$\text{prev}[i] = \begin{cases} \max\{j \mid j < i,\ A[j] = A[i]\} & \text{if such } j \text{ exists} \\ -1 & \text{otherwise} \end{cases} \tag{1}$$

The distinct count becomes:

$$\text{distinct}(L, R) = \sum_{i=L}^{R} \mathbf{1}[\,\text{prev}[i] < L\,] \tag{2}$$

This reformulation replaces frequency bookkeeping with a sequential scan that is SIMD- and GPU-friendly [1].

**Algorithm 1** Prev-Array Distinct Counting (CPU/GPU core)

```
Compress array values to [0, σ)
Compute prev[i] = previous index of A[i], or −1
for each query [L, R] do
    cnt ← 0
    for i = L to R do
        if prev[i] < L then
            cnt ← cnt + 1
        end if
    end for
    output cnt
end for
```

## 6.2 Engine 1: Mo's Algorithm with Hilbert Ordering

Queries are sorted by Hilbert curve order over $(L, R)$ to minimize window movement and maximize cache locality [2, 3, 4, 5].

## 6.3 Engine 2: OpenMP + SIMD

Queries are distributed across threads with `#pragma omp parallel for` and reduced with `#pragma omp simd` to exploit AVX-class SIMD lanes [6, **?**, 7, 8]. Parallel efficiency is measured as:

$$E(p) = \frac{T_1}{p \cdot T_p} \tag{3}$$

## 6.4 Engine 3: CUDA Warp-Level Parallelism

One warp (32 threads) processes a query with stride-32 scanning and warp-shuffle reduction, minimizing synchronization overhead and maximizing coalesced access [**?**, 9, 10].

## 6.5 Benchmarking Protocol

Datasets are generated once and reused across engines. Timings capture query processing time, and GPU runs additionally decompose host-to-device, kernel, and device-to-host phases. Results are exported to `results/data.csv`. Binary inputs reduce parsing overhead to keep benchmarks compute-bound [11].

# 7 Datasets

All datasets are synthetically generated with a fixed seed for reproducibility. The generator (`input_generator`) accepts the parameters below.

Table 2: Dataset Generation Parameters

| **Parameter** | **Value** | **Description** |
|---|---|---|
| `--n` | 1,000,000 | Length of the input array $A$ |
| `--q` | 100,000 | Number of offline queries |
| `--value-range` | 1,000,000 | Values drawn from $[0, V)$ |
| `--seed` | 42 | PRNG seed for reproducibility |
| `--out-dir` | `results/` | Output directory |

**Binary Format.** Each binary file stores a 32-bit count header followed by packed 32-bit integers or query structs (little-endian). This eliminates text parsing overhead and keeps benchmarks compute-bound [11].

Table 3: Datasets

| **Name** | **Description** | **Size** | **Year** | **Organization** |
|---|---|---|---|---|
| array.bin | Uniform random integers, value_range $= 10^6$ | 1,000,000 ints | 2026 | Project_SS |
| queries_small.bin | $Q = 100{,}000$, max_len $= 128$ | 100,000 ranges | 2026 | Project_SS |
| queries_medium.bin | $Q = 100{,}000$, max_len $= 2048$ | 100,000 ranges | 2026 | Project_SS |
| queries_large.bin | $Q = 100{,}000$, max_len $= 32768$ | 100,000 ranges | 2026 | Project_SS |
| results/data.csv | Aggregated metrics (query_sec, throughput, speedup, efficiency) | 31 rows | 2026 | Project_SS |

# 8 Results and Discussion

## 8.1 Execution Time

Execution time is shown on a log scale in Figure 3. The Hilbert-ordered Mo baseline performs well in single-threaded mode due to reduced window movement, while OpenMP and CUDA accelerate the prev-array scan by exploiting SIMD and warp-level parallelism.

## 8.2 Throughput and Speedup

At $Q = 100{,}000$ with medium ranges (max_len=2048), Mo's achieves query_sec 0.040219 (throughput 2.49M qps). OpenMP (20 threads) reduces query_sec to 0.004766 for 8.44× speedup and 20.98M qps. CUDA achieves query_sec 0.001170 for 34.37× speedup and 85.47M qps. Figure 2 summarizes the relative speedup at the largest workload.

Table 4: Summary at $Q = 100{,}000$ (medium ranges, max_len=2048)

| Method | Query time (s) | Throughput (qps) | Speedup |
|---|---|---|---|
| Mo's | 0.040219 | 2,486,393 | 1.00× |
| OpenMP (20 threads) | 0.004766 | 20,981,295 | 8.44× |
| CUDA | 0.001170 | 85,468,624 | 34.37× |

## 8.3 Range Sensitivity

Larger query windows reduce throughput on both CPU and GPU. At $Q = 50{,}000$ with large ranges, query_sec rises to 0.077011 (Mo), 0.030079 (OpenMP), and 0.003874 (CUDA), but the GPU remains dominant.

## 8.4 Thread Efficiency (OpenMP)

Efficiency improves quickly up to 8 threads and then tapers as memory bandwidth becomes the bottleneck. At 16 threads, the observed speedup is 5.18 with efficiency 0.324.

## 8.5 GPU Transfer/Kernel Breakdown

For $Q = 100{,}000$, host-to-device transfer is 0.678 ms, kernel time is 0.424 ms, and device-to-host is 0.068 ms, showing that kernel execution dominates once data is resident.

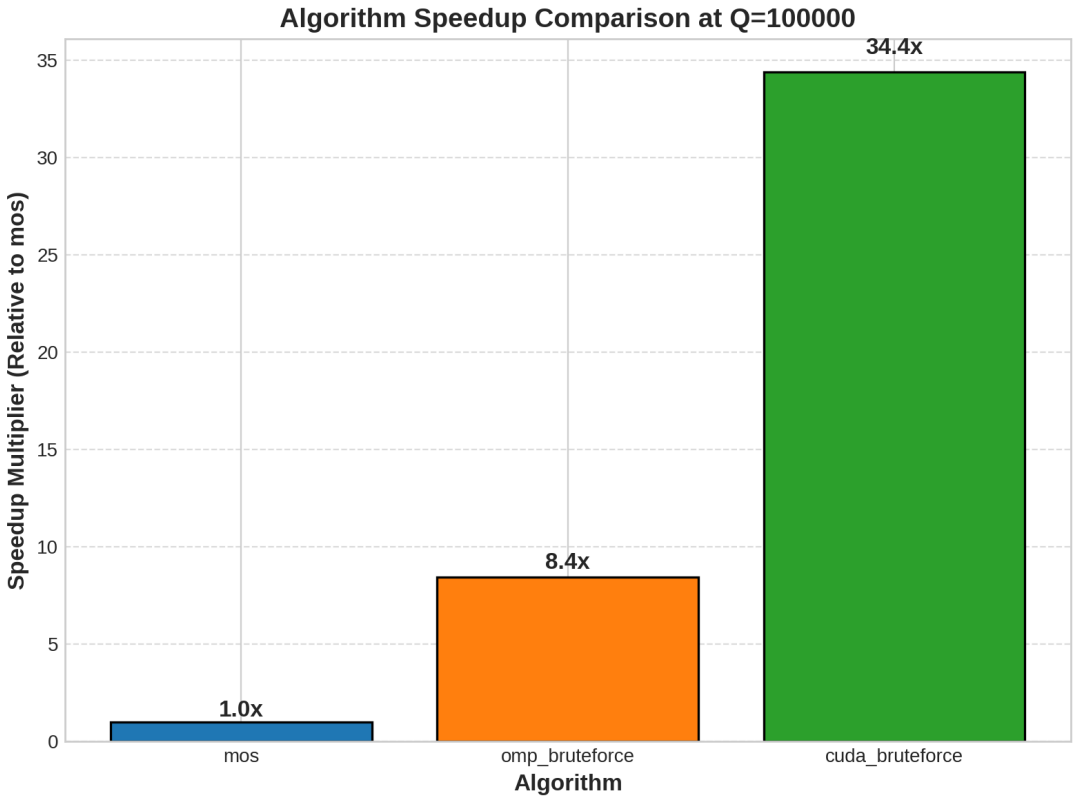


Figure 2: Speedup comparison at $Q = 100{,}000$ (medium ranges).

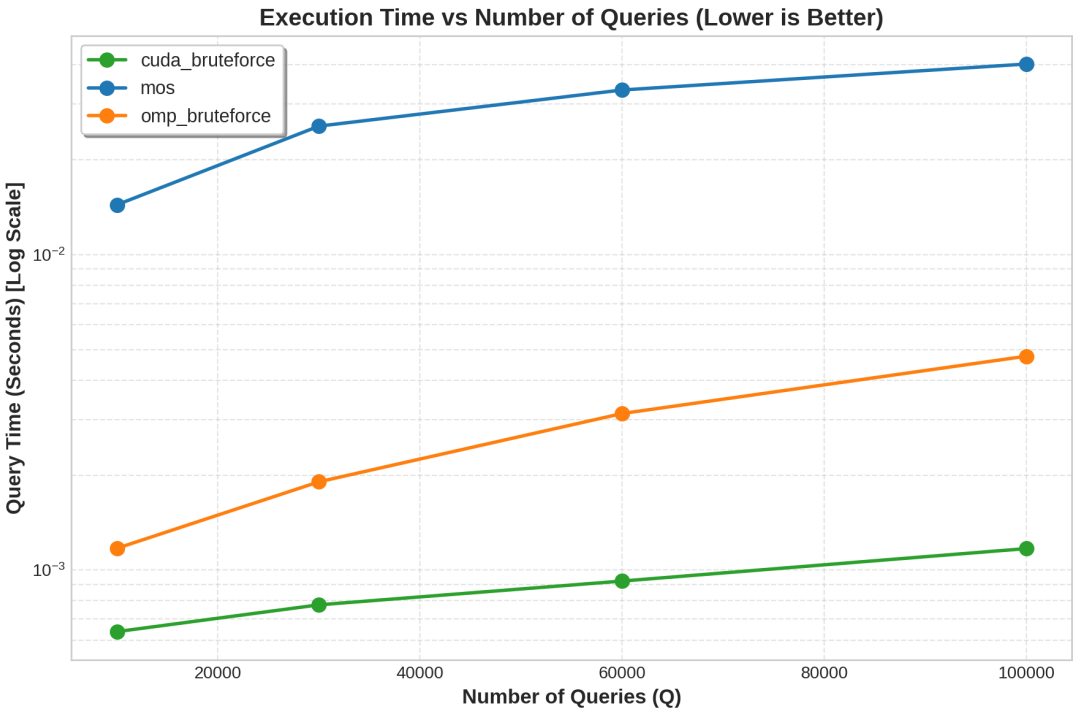


Figure 3: Execution time vs. number of queries (log scale).

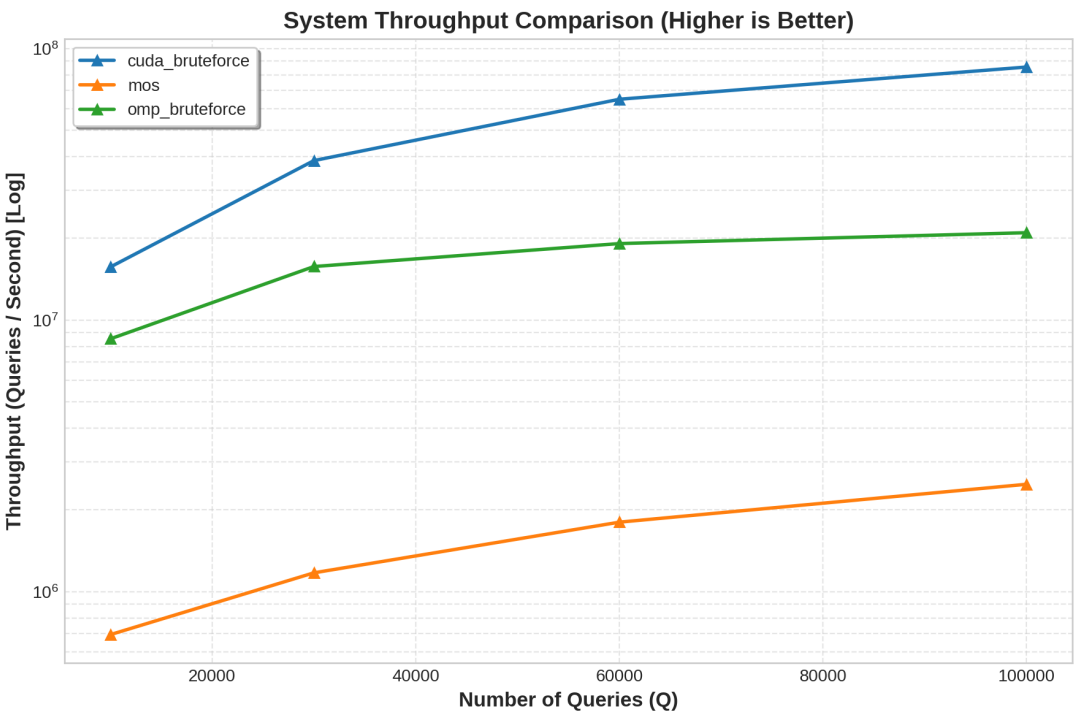


Figure 4: Throughput comparison across methods (log scale).

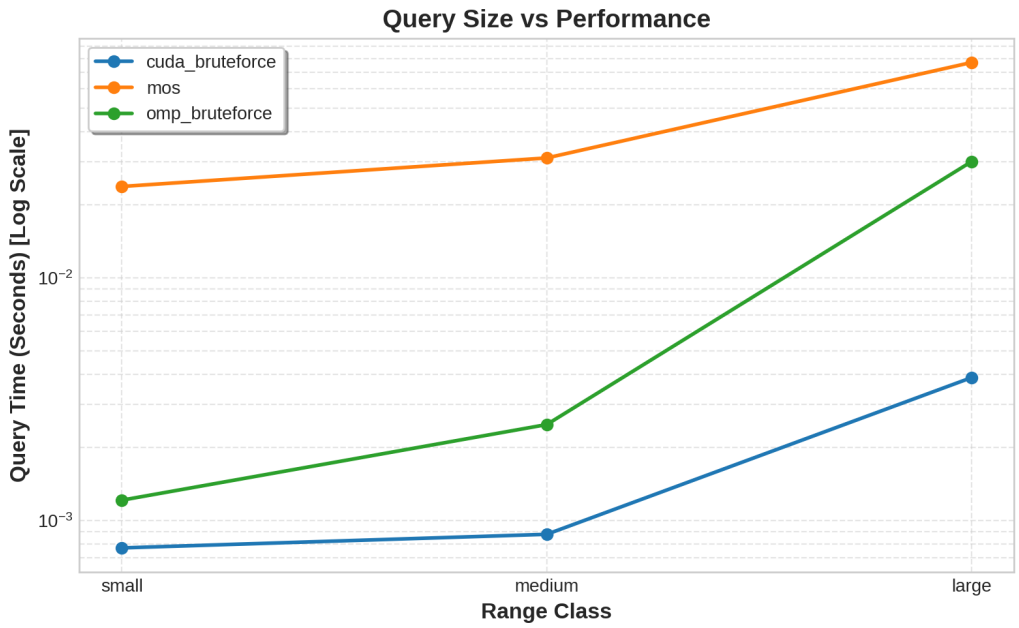


Figure 5: Query size vs. performance across range classes.

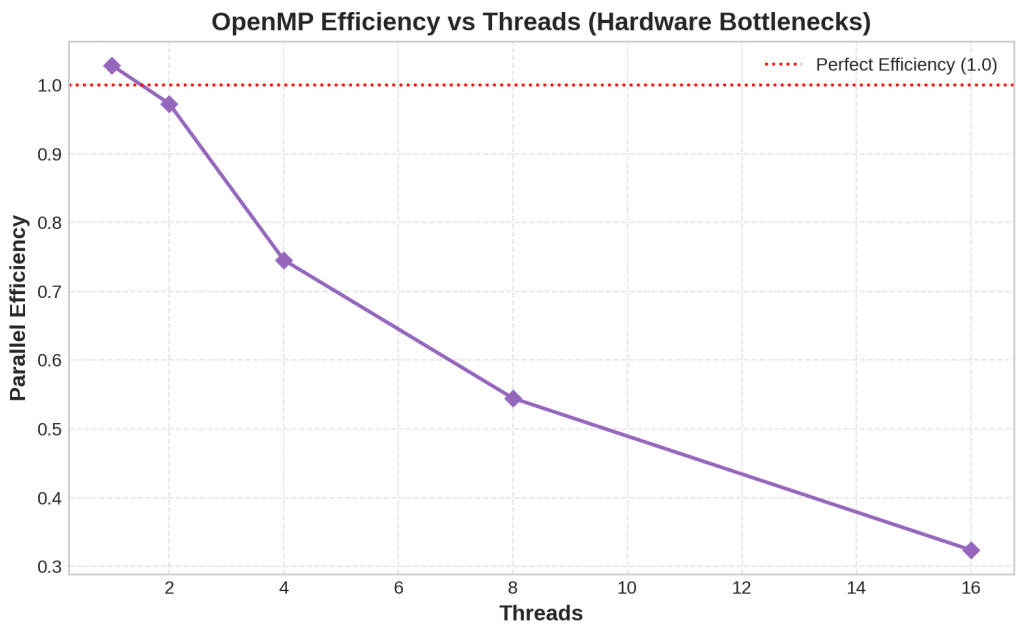


Figure 6: OpenMP efficiency vs. thread count.

# References


[1] X. Deng *et al.*, "Offline range query algorithms with previous-occurrence transformations," in *Proceedings of the International Conference on Data Engineering (ICDE)*, 2019.

[2] T. S. Mo, "Mo's algorithm for offline range queries," 1974, attributed; formal publication varies by source.

[3] D. Hilbert, "Ueber die stetige abbildung einer linie auf ein flaechenstueck," *Mathematische Annalen*, vol. 38, pp. 459–460, 1891.

[4] B. Moon, H. V. Jagadish, C. Faloutsos, and J. H. Saltz, "Analysis of the clustering properties of the hilbert space-filling curve," *IEEE Transactions on Knowledge and Data Engineering*, vol. 13, no. 1, pp. 124–141, 2001.

[5] H. Haverkort and F. van Walderveen, "Four-dimensional hilbert curves for r-trees," *Journal of Experimental Algorithmics*, vol. 16, p. 3, 2011.

[6] L. Dagum and R. Menon, "Openmp: An industry-standard api for shared-memory programming," *IEEE Computational Science & Engineering*, vol. 5, no. 1, pp. 46–55, 1998.

[7] Intel Corporation, *Intel Intrinsics Guide: AVX2 and AVX-512*, 2023. [Online]. Available: https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html

[8] M. Kruse and H. Finkel, "A proposal for loop-transformation pragmas," in *Evolving Enhancements to OpenMP*, ser. LNCS, vol. 13071, 2021, pp. 37–52.

[9] M. Harris, S. Sengupta, and J. D. Owens, "Parallel prefix sum (scan) with cuda," in *GPU Gems 3*. NVIDIA, 2007, ch. 39.

[10] D. Merrill and M. Garland, "Single-pass parallel prefix scan with decoupled look-back," NVIDIA, Tech. Rep. NVR-2016-002, 2016.

[11] D. Lemire *et al.*, "Parsing gigabytes of json per second," *The VLDB Journal*, vol. 28, no. 6, pp. 941–960, 2019.